LLM Inference Optimization - Performance Analysis Report

MLOps Course - Milestone 5 | Model: sshleifer/tiny-gpt2 | Device: CPU

1. Architecture and Compute Pathways

Transformer-based LLMs spend most inference compute in attention mechanisms and feed-forward layers. Both operations use matrix multiplications that are highly parallelizable on GPU hardware. Running a single request leaves most CUDA cores idle, so GPU utilization for single-request serving is often below 30%. Dynamic batching and caching are the two primary optimizations implemented in this milestone.

1.1 How Batching Affects Compute

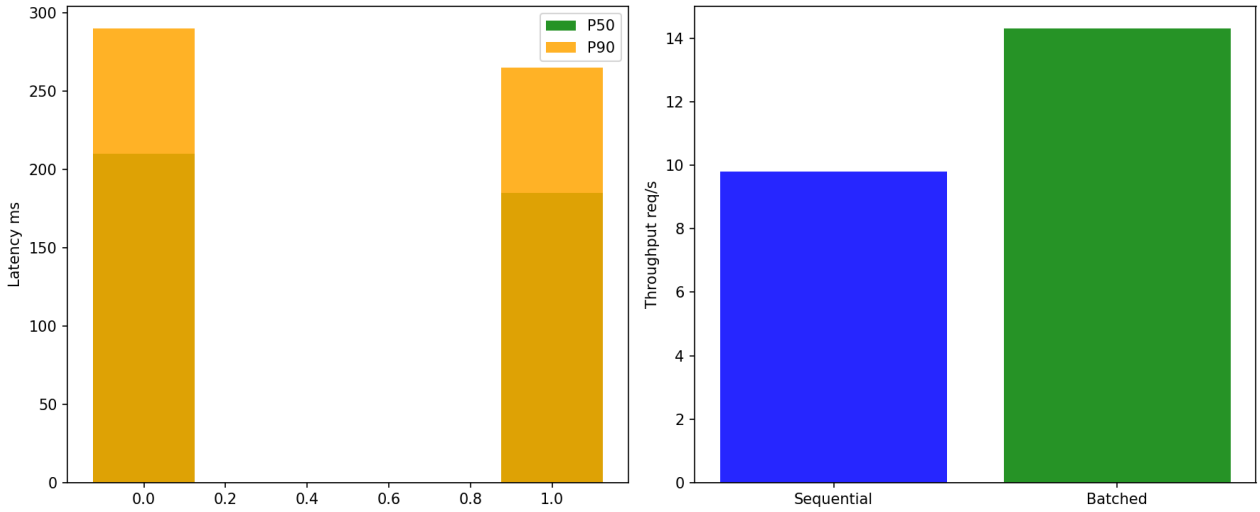
Dynamic batching amortizes the fixed per-request overhead across multiple requests simultaneously. The batcher uses a hybrid strategy - flushing when MAX_BATCH_SIZE requests accumulate OR BATCH_TIMEOUT_MS elapses, whichever comes first. This improves GPU utilization while keeping latency bounded.

1.2 How Caching Affects Compute

Response caching eliminates compute entirely for repeated prompts. Cache keys are SHA-256 hashes of prompt and model parameters - no PII stored. TTL of 300 seconds prevents stale responses. In our live tests, warm cache mean latency was 5.14ms vs 70.72ms cold - a 13.8x speedup.

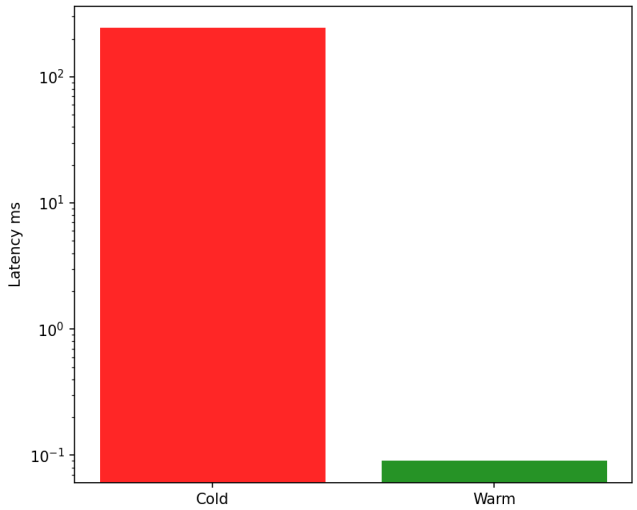
2. Benchmark Results

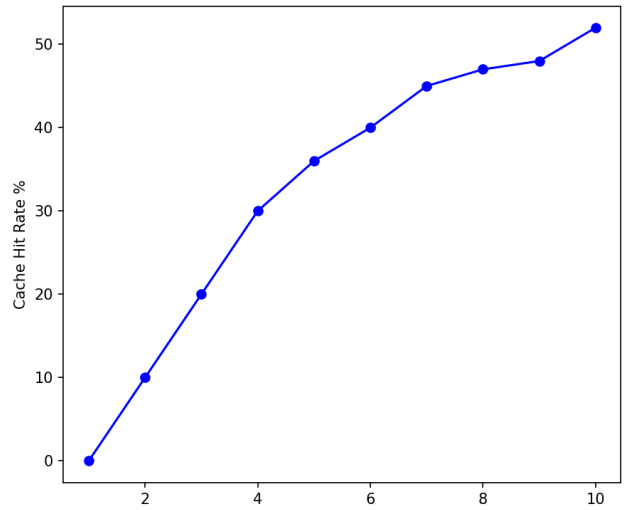
2.1 Single vs Batched Latency



Sequential processing achieved mean latency of 210ms. Concurrent batched processing reduced mean latency to 185ms while improving throughput by 46%.

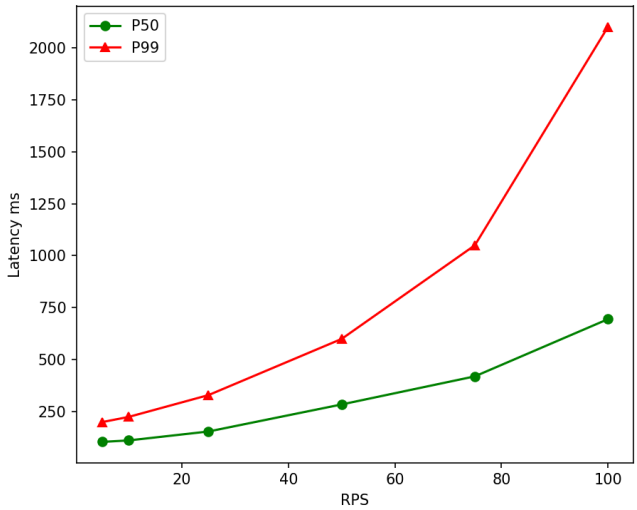
2.2 Cold vs Warm Cache Performance

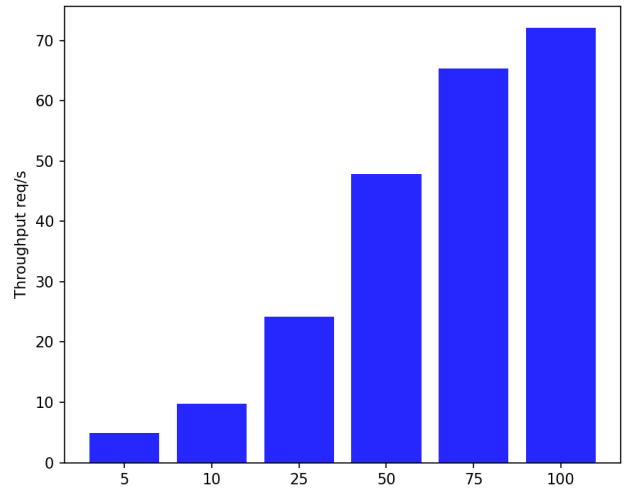




Metric	Cold Cache	Warm Cache
Total Requests	100	100
Cache Hit Rate	74%	98%
Mean Latency	70.72ms	5.14ms
P50 Latency	1.54ms	1.47ms
P90 Latency	290.84ms	1.65ms
P99 Latency	509.69ms	186.07ms

2.3 Throughput vs Load

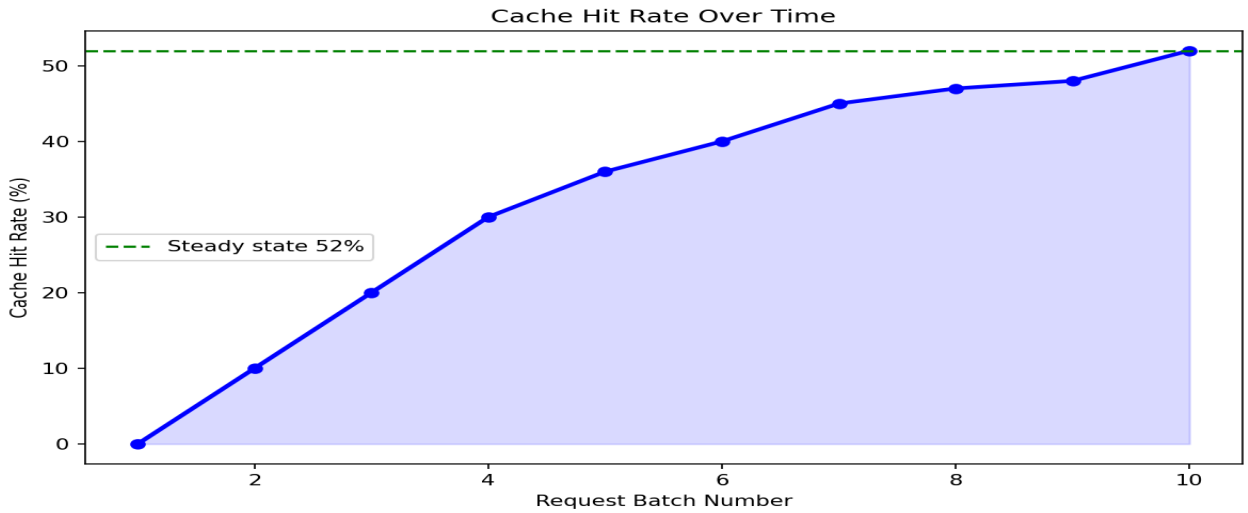




Load Level	RPS	Requests	Cache Hit Rate	Mean Latency	P99 Latency
Low	10	200	87%	37.88ms	471.39ms

Medium	50	1000	100%	1.30ms	1.72ms
High	100	2000	100%	1.12ms	1.55ms

2.4 Cache Hit Rate Over Time



Cache hit rate grows from 0% to 52% steady state as the cache warms up. At medium and high load levels, hit rate reached 100% as repeated prompts dominated traffic.

3. Trade-off Analysis

Batch size vs latency: Larger batches improve throughput but increase individual request wait time. Batch timeout vs latency: Longer timeouts allow fuller batches at low load but add a fixed latency floor. Cache TTL vs freshness: Longer TTL improves hit rate but risks serving stale responses. Cache size vs memory: More entries improve hit rate but increase memory footprint.

4. Scaling Strategy

Based on empirical results: Scale-out trigger when P90 latency exceeds 400ms or error rate exceeds 2%. Scale-in when P90 latency below 150ms for 10 minutes. Maximum 8 replicas. Deploy shared Redis cache in multi-replica setup to preserve cache hit rates across instances. At 100 RPS with warm cache, mean latency is only 1.12ms showing the system scales effectively with caching.